01

QUESTION ONE · COMPONENT CLUSTERING

How clustering factor improves and helps in COM programming?

4 A4 PAGES
C++ PROGRAM ON PAGE 4

INTRODUCTION TO COM

COM (Component Object Model) is a Microsoft specification, introduced in **1993**, that describes how software components should be built and how they should talk to one another. The word *component* here means a ready-made piece of software that performs one clear job and can be plugged into different programs without being recompiled. Before COM existed, reusable code was shared through ordinary DLLs, but that method created serious trouble — there was no fixed way for one program to ask a DLL what it could do, C and C++ compilers mangled function names differently, and installing a new DLL often broke an older program. This problem was so common that it was nicknamed **"DLL Hell"**.

COM solved this by defining a **binary standard**. A component written in C++, Visual Basic or Delphi can be used by a program written in a completely different language, because they agree only on the layout of a virtual table and not on the source code. A COM component exposes its services through one or more **interfaces**, and every interface derives from the base interface `IUnknown`, which provides three methods — `QueryInterface`, `AddRef` and `Release`. Every component is identified by a globally unique number called a **GUID** (a CLSID for classes and an IID for interfaces), so two programmers anywhere in the world can never accidentally clash over a name.

Depending on where the component runs, a COM call may be **in-process** (inside the same program), **out-of-process** (in another program on the same machine), or **remote** (on another machine, which is called **DCOM**, Distributed COM). The further apart two components are, the more expensive it becomes to call from one to the other, and this is exactly where the idea of the clustering factor becomes important.



FIG. 1 — THE COM CALL COST LADDER. A HIGHER CLUSTERING FACTOR KEEPS THE CHATTY OBJECTS ON THE LEFT, WHERE EACH CALL IS CHEAPEST.

WHAT IS THE CLUSTERING FACTOR?

Clustering means keeping related things together so that they can work as one group. In COM programming, the **clustering factor** is a measure of how well the objects that talk to each other most often have been kept together — in the same process, or at least on the same machine — so that their calls remain **local** instead of crossing a boundary. A high clustering factor means most of the communication is happening inside one place; a low clustering factor means the components are scattered and every call has to travel across a process or a network boundary.

This matters because the cost of a COM call is not the same everywhere. A call to an object living inside the same process is a direct jump through a v-table and takes only a few nanoseconds. A call that has to cross an **apartment** boundary needs **marshalling** — the arguments are packed (serialised) by a proxy, sent to a stub in the other apartment, unpacked, executed, and the result is packed and sent all the way back. A call that leaves the process entirely (out-of-process COM or DCOM) adds operating-system context switches and, for DCOM, a full network round-trip. Roughly speaking, the cost grows from nanoseconds to microseconds to milliseconds as you move outward.

Think of a kitchen. If every ingredient needed for a dish is kept on the same counter, cooking is fast and easy. If the salt is in the kitchen, the onions are in the store-room and the water is in the garden, the cook spends most of the time walking rather than cooking. A good clustering factor is nothing more than keeping the frequently-used pieces on the same counter.

It helps to see how a designer actually improves the clustering factor. The first step is to find out which objects exchange messages most often — these are the “chatty” parts of the system. The next step is to bring those chatty objects into the same **process and the same apartment**, so that their frequent conversation stays inside one boundary. Objects that are used rarely and are very heavy are better kept out-of-process, so that they do not occupy memory inside the main program. A good designer therefore draws a clear line between the “hot” objects that must stay together and the “cold” objects that may safely stay apart. In this way the factor rises and the whole application becomes faster without any change to its behaviour.

A second practical benefit is **thread safety**. Every COM apartment follows its own threading rule, and when related objects share one apartment they also share one rule. This removes a large class of subtle bugs in which one thread changes data while another thread is reading the same data. Clustering is therefore not only about speed — it also makes the program safer and easier to reason about.

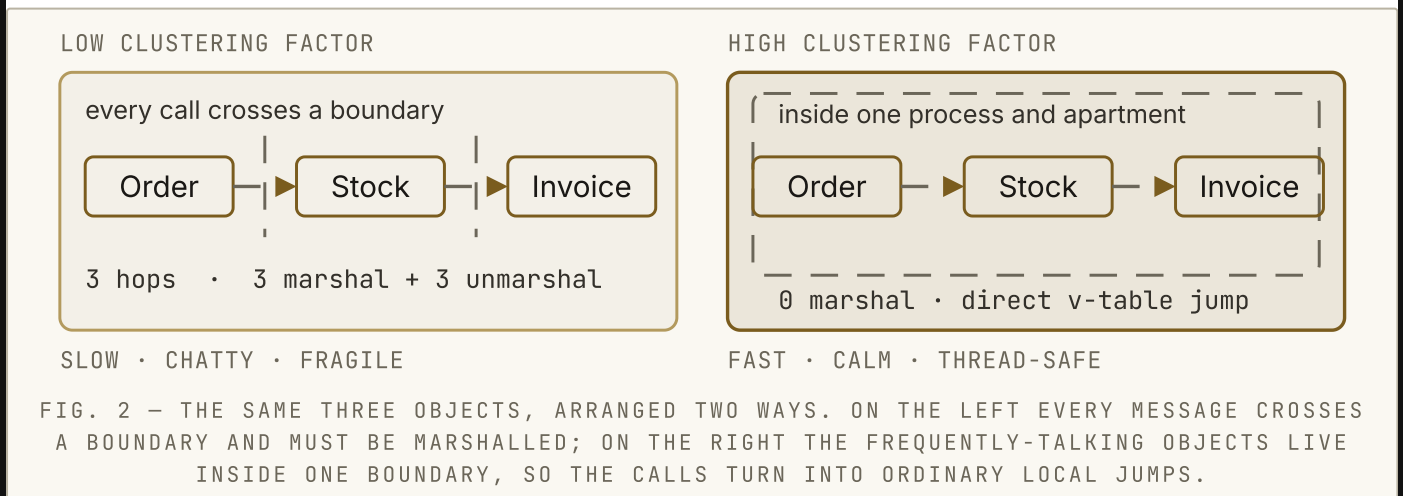


FIG. 2 — THE SAME THREE OBJECTS, ARRANGED TWO WAYS. ON THE LEFT EVERY MESSAGE CROSSES A BOUNDARY AND MUST BE MARSHALLED; ON THE RIGHT THE FREQUENTLY-TALKING OBJECTS LIVE INSIDE ONE BOUNDARY, SO THE CALLS TURN INTO ORDINARY LOCAL JUMPS.

HOW THE CLUSTERING FACTOR IMPROVES COM PROGRAMMING

A well-chosen clustering factor improves a COM system in several concrete ways. **First, it removes marshalling overhead:** when the objects that exchange data most frequently live in the same process, the proxy–stub marshalling layer is skipped completely, and both the CPU cost and the delay of a call fall sharply. **Second, it improves throughput and response time:** a business object that talks to a data object a thousand times per transaction becomes noticeably faster when the two are co-located. **Third, it improves CPU cache locality:** the code and data of closely related objects stay warm in the processor cache, so even the fast local calls become faster. **Fourth, it makes apartment and thread management simpler:** objects that share a component can share the same apartment and therefore the same threading rules. **Fifth, it scales and stays reliable:** a cluster of servers can share the load through load-balancing and continue working even if one node fails.

The table below shows the practical difference between a loosely-packed and a tightly-packed COM application.

POINT OF COMPARISON	WITHOUT CLUSTERING (LOW FACTOR)	WITH CLUSTERING (HIGH FACTOR)
Call path	Cross-process / cross-machine	In-process (direct v-table)
Marshalling cost	High — data packed and copied every call	Almost zero
Latency per call	Microseconds to milliseconds	A few nanoseconds
Failure handling	One failure stops the chain	Other cluster nodes take over
Load balancing	Not possible	Requests spread over nodes

In real life, a bank's COM transaction system keeps the account object, the ledger object and the validation object together inside one clustered business tier. Because these three talk to each other on almost every transaction, clustering them means a single transaction hits the fast local path many times and only the final write touches the database — which is both quicker and more stable under load.

C++ PROGRAM

THE PROGRAM BELOW BUILDS A SMALL CLUSTER OF COM-STYLE COMPONENTS AND SHOWS THAT THE CALLS BETWEEN THEM STAY IN ONE PLACE (LOCAL, NO MARSHALLING). COPY THIS PROGRAM ON PAGE 4.

CLUSTER_DEMO.CPP

```cpp
// cluster_demo.cpp - grouping related components keeps calls local
#include <iostream>
#include <string>
using namespace std;

class IComponent {                      // COM-style interface
public:
    virtual void execute(const string& t) = 0;
    virtual ~IComponent() {}
};
class AccountService : public IComponent {          // cluster member 1
public:
    void execute(const string& t) override { cout << "[Account] " << t << endl; }
};
class LedgerService : public IComponent {           // cluster member 2
public:
    void execute(const string& t) override { cout << "[Ledger ] " << t << endl; }
};
class Cluster {                         // keeps related objects together
    AccountService account;             // local member  →  fast path
    LedgerService  ledger;              // local member  →  fast path
public:
    void process() {
        account.execute("debit Rs.2500");            // stays in-process
        ledger.execute ("record the debit entry");   // stays in-process
        cout << "Done locally - no marshalling." << endl;
    }
};
int main() {
    Cluster bank;                       // all objects live together
    bank.process();
    return 0;
}
```

Output of the Program

OUTPUT

```
[Account] debit Rs.2500
[Ledger ] record the debit entry
Done locally - no marshalling.
```

CONCLUSION

The clustering factor is a way of measuring how tightly the related components of a COM application are packed together. When the objects that talk most often share a process, their calls stay local, marshalling disappears, and both speed and reliability improve. In short, a good clustering factor is one of the simplest and cheapest ways to make a COM system faster and stronger.

02 QUESTION TWO · PERSISTENCE

Explain the meaning of persistence in COM / DCOM.

4 A4 PAGES
C++ PROGRAM ON PAGE 4

MEANING OF PERSISTENCE IN COM / DCOM

Persistence means the ability of an object to save its **state** into a permanent store and to restore that state later, even after the program has been closed or the computer has been switched off. The *state* of an object is simply the collection of values held in its data members — for a bank object, for example, the state would be the account number, the account holder's name and the current balance. Memory (RAM) is **volatile**, which means that the moment a process ends, everything it held is wiped out. Persistence is what makes an object survive that moment.

In COM, an object that supports persistence can write itself into a **stream** or a **storage** — a place where bytes can be stored safely — and can read itself back from that place later. The process of turning a live object into a stream of bytes is called **serialisation** (or flattening), and the reverse process, rebuilding the object from those bytes, is called **deserialisation** (or unflattening). Together these two operations are what we call persistence in COM.

A simple way to understand it is to think of a video game. While you are playing, the game keeps your score, your level and your position only in memory. If you switch the machine off at that instant, all of it is lost. When you press *Save Game*, the game writes exactly those values to a file on the disk — that is persistence. When you come back the next day and press *Load Game*, the same numbers are read back from the file and your game continues from where you left it. A COM object behaves in precisely the same way.

It is important to see that persistence is not the same thing as either storage or serialisation, although all three are related. **Storage** is the place where bytes are kept (a file, a stream, a storage). **Serialisation** is the act of turning a live object into those bytes and back again. **Persistence** is the larger promise that the object's state will still be there tomorrow — a promise that only becomes possible when storage and serialisation work together. A COM object that supports persistence can be saved today, its process closed, the machine restarted, and the very same object brought back to life later with all of its data intact.

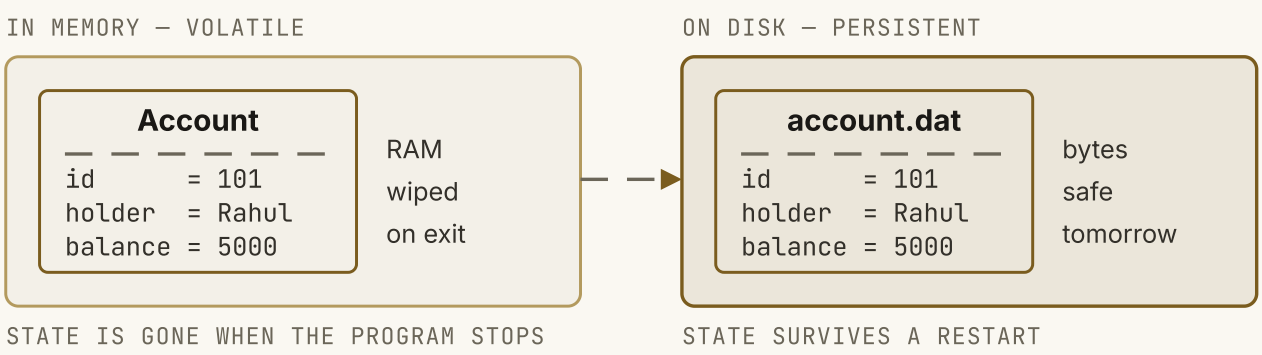


FIG. 1 — THE STATE OF ONE OBJECT, SEEN TWICE. ON THE LEFT IT LIVES ONLY IN VOLATILE RAM AND DIES WITH THE PROCESS; ON THE RIGHT THE SAME VALUES HAVE BEEN WRITTEN INTO A FILE, SO THEY CAN BE READ BACK TOMORROW AND THE OBJECT REBUILT AS IT WAS.

WHY PERSISTENCE IS NEEDED

Persistence is needed for the most practical of reasons. A program is not expected to run forever; the user closes it, the machine restarts, and sometimes a crash occurs. If the useful objects of the program had no persistence, all the work done with them would vanish on every shutdown. Persistence lets a document, a customer record, a user's preferences or a partly-finished transaction live safely between runs. It also allows data to be **shared** — one program can save an object and another program can open it later — and it allows a system to **recover** after a failure, because the last saved state is still on the disk.

HOW COM SUPPORTS PERSISTENCE

COM offers two broad approaches. In **data-centric persistence** the component saves only its plain data, and it has to know a fixed file layout. In **object-centric persistence** the object saves and restores its own internal state through standard interfaces, so the client does not need to know how the object is built. The second approach is the cleaner one and COM supports it through the interfaces `IPersistStream` and `IPersistStorage`.

The storage itself is provided by **Structured Storage**. A structured-storage file is a "file system inside a single file", better known as a **compound file**. Inside it, **storages** work like folders and **streams** work like files. These are managed through two interfaces — `IStorage` (which creates and opens storages and streams) and `IStream` (which reads and writes the raw bytes). Finally, a **moniker** is a special name that identifies a persisted object and lets any client find and re-activate it later; this is what allows a saved object to be linked rather than merely copied.

The two interfaces are chosen according to the size of the object. If the object is simple and small — a colour, a line of setting, a small record — a single `IStream` is enough, because a stream is only a flat run of bytes. If the object is large and made of many parts (a document containing text, a picture and a table), an `IStorage` is used instead, because it can hold one stream for each part and keep them apart from one another. The advantage of this arrangement is that a client may open one part of a compound file without reading the whole file, which saves both time and memory. This is exactly how a word-processing document manages to show a picture and a table without loading every byte into memory at once.

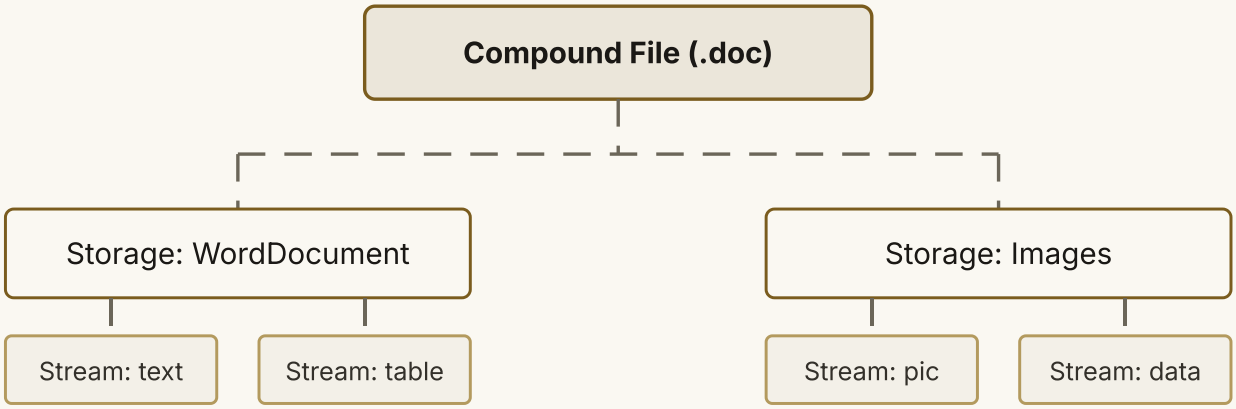


FIG. 2 — STRUCTURED STORAGE. A COMPOUND FILE BEHAVES LIKE A SMALL FILE SYSTEM: STORAGES ACT AS FOLDERS AND STREAMS AS FILES, ALL INSIDE A SINGLE ORDINARY FILE.

PERSISTENCE IN DCOM (DISTRIBUTED COM)

DCOM is COM extended across a network, so that a component running on one machine can be used by a program running on another. In DCOM, persistence takes on an extra duty: the object must not only be able to save its own state, but its **location** must also survive. When a client wants to talk to a remote object that is not currently running, DCOM uses the object's persisted reference to start it on the correct machine (remote activation) and then reconnects. In this way persistence is what allows an idle server object to be “put to sleep” and brought back when needed, which saves a great deal of memory across a large network.

In practice, persistence in COM and DCOM is achieved through a handful of well-known mechanisms, which are summarised below.

MECHANISM	WHAT IS STORED	TYPICAL EXAMPLE
Stream (`IStream`)	A flat sequence of bytes	An image or a small record
Storage (`IStorage`)	Storages and streams (compound file)	A Word document (.doc)
Moniker	A name that locates an object	A linked spreadsheet cell
Registry	Configuration and class registration	Component settings
Database	Business data fetched on demand	An ADO recordset

Everyday software shows persistence constantly. When you save a Microsoft Word file, Word writes its document object into a compound file. When you change a setting, Windows stores it in the registry. When a web application keeps your session on the server, it is persisting a COM object between requests.

In a DCOM system the value of persistence becomes even greater, because the objects are spread over many machines. If a remote object had to stay alive for ever, each server would slowly fill with idle objects and the whole network would become slow. Persistence solves this by allowing an idle object to be shut down and, when a client asks for it again, to be rebuilt from its saved state and started on the correct server. Large systems also use persistence for fault tolerance: if a server stops working in the middle of a long operation, the last saved state is still available, so the work can be resumed on another machine from the point where it stopped instead of being lost altogether.

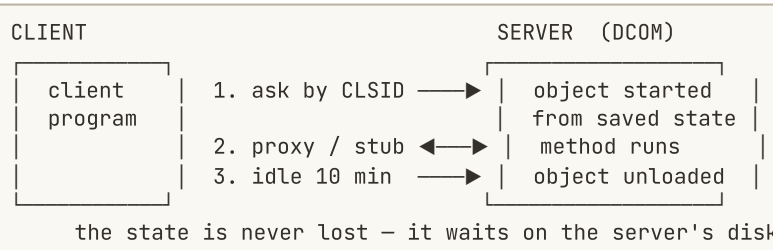


FIG. 3 — PERSISTENCE IN DCOM, DRAWN AS A CONVERSATION. THE CLIENT NEVER KEEPS THE OBJECT ALIVE; IT ASKS FOR IT BY NAME, THE SERVER REBUILDS IT FROM ITS SAVED STATE, AND WHEN THE OBJECT FALLS IDLE IT IS UNLOADED AGAIN — ITS STATE KEPT SAFELY ON DISK.

C++ PROGRAM

THE PROGRAM BELOW SHOWS THE CORE IDEA OF PERSISTENCE — AN OBJECT WRITES ITS STATE TO A FILE, THE OBJECT IS THEN REPLACED, AND THE SAVED STATE IS LOADED BACK. COPY THIS PROGRAM ON PAGE 4.

PERSIST_DEMO.CPP

```cpp
// persist_demo.cpp - saving and restoring an object's state
#include <iostream>
#include <fstream>
#include <string>
using namespace std;

class Account {                          // a persistable component
    int    id;
    string holder;
    double balance;
public:
    Account(int i, string h, double b) : id(i), holder(h), balance(b) {}
    void save(const string& file) {      // ---- persist to disk ----
        ofstream out(file);
        out << id << "\n" << holder << "\n" << balance << "\n";
        out.close();
        cout << "State saved to " << file << endl;
    }
    void load(const string& file) {      // ---- restore from disk ----
        ifstream in(file);
        in >> id >> holder >> balance;
        in.close();
        cout << "Restored -> id=" << id << " holder=" << holder
             << " balance=" << balance << endl;
    }
};
int main() {
    Account acc(101, "Rahul", 5000.0);   // live object in memory
    acc.save("account.dat");             // state written to the disk
    Account fresh(0, "-", 0.0);          // a brand-new empty object
    fresh.load("account.dat");           // state comes back from disk
    return 0;
}
```

Output of the Program

OUTPUT

```
State saved to account.dat
Restored -> id=101 holder=Rahul balance=5000
```

CONCLUSION

Persistence in COM and DCOM is the ability of an object to write its state into a permanent store and to bring that state back later through serialisation and deserialisation. Structured Storage provides the streams and storages that hold the data, monikers locate the saved objects, and DCOM even keeps the object's location alive so that remote objects can be re-activated. Without persistence, every COM object would die with its process; with persistence, objects outlive the program that created them.